# ANÁLISIS DE COMPLEJIDAD Y DESEMPEÑO

## Algoritmos CSP y Búsqueda Adversaria

**Integrantes:**

Alejandro Jaramillo

Danial Diab

Gustavo Candela

Santiago Castro

## PUNTO A: ANÁLISIS CSP

### 1. COMPLEJIDAD ESPACIAL

Sea n el número de variables (puntos de entrega), d el tamaño máximo del dominio (drones disponibles) y c el número de restricciones del problema (capacidad del dron, batería disponible, ventanas de tiempo y conflictos de asignación).

En el peor caso, sin optimizaciones, el problema de asignación puede requerir explorar todas las posibles combinaciones de valores para las variables, lo que produce una complejidad espacial y temporal de $O(d^n)$.

**Backtracking básico**

El algoritmo de backtracking_search explora el árbol de búsqueda probando valores para cada variable y verificando si las restricciones se cumplen.

- ALGORITMO: backtracking_search
  Espacio: O(d^n) - árbol completo Stack recursion: O(n)
  Dominios: O(n*d)

Este algoritmo no realiza poda temprana, por lo que puede explorar gran parte del espacio de búsqueda.

**Backtracking con Forward Checking**

El algoritmo backtracking_fc introduce un mecanismo de poda temprana eliminando valores inconsistentes de los dominios de variables futuras.

- ALGORITMO: backtracking_fc (Forward Checking)

Espacio: O(n*d^2) - dominios reducidos
Mejora: Detecta falla temprano, poda ~40-60% espacio

Esto ocurre porque cada asignación requiere revisar los dominios restantes para verificar consistencia con las restricciones.

La principal ventaja de Forward Checking es que permite detectar inconsistencias antes de expandir ramas completas del árbol de búsqueda.

**Backtracking con Arc Consistency (AC-3)**

El algoritmo backtracking_ac3 utiliza el procedimiento AC-3 para mantener consistencia entre pares de variables.

- ALGORITMO: backtracking_ac3 (Arc Consistency)
  Espacio: O(n^2*d^3) - cola arcos
  Mejora: Consistencia global antes backtracking

Esto se debe al almacenamiento de la cola de arcos y a las revisiones necesarias para garantizar consistencia entre variables conectadas por restricciones.

Este enfoque es especialmente útil en problemas con muchas restricciones, ya que elimina valores inconsistentes antes de iniciar el proceso de backtracking.

**Backtracking con heurísticas MRV y LCV**

El algoritmo backtracking_mrv_lcv incorpora dos heurísticas importantes:

MRV (Minimum Remaining Values): selecciona la variable con menor número de valores posibles.

LCV (Least Constraining Value): selecciona el valor que menos restringe a las demás variables.

- ALGORITMO: backtracking_mrv_lcv
  Espacio: O(n*d^2) - heurística variable ordering
  Mejora: MRV reduce branching factor significativamente

Estas heurísticas no cambian el peor caso teórico del problema, pero reducen significativamente el factor de ramificación efectivo del árbol de búsqueda.

## 2. COMPLEJIDAD TEMPORAL

**Backtracking básico**

- backtracking_search: O(d^n)

Peor caso: explora todos nodos del árbol
Ejemplo: 5 entregas, 3 drones = 3^5 = 243 nodos

**Backtracking con Forward Checking**

- backtracking_fc: O(n·d^3) por nodo
  Checklist forward_check: ~O(n·d^2)
  Típico: 90% menos nodos que backtracking puro

**Backtracking con AC-3**

- backtracking_ac3: O(e·d^3) preproceso + backtracking
  e = número arcos  n^2
  Mejora: 95% nodos en problemas tight

**Backtracking con MRV y LCV**

- backtracking_mrv_lcv: O(n^2·d^4) peor caso
  MRV heuristic: O(n) por variable selection
  LCV heuristic: O(d·n) conflicts por valor
  Value ordering reduce costo real significativamente

## 3. REDUCCION ESPACIO: Análisis Cuantitativo

- Búsqueda Exhaustiva Simple:
  Asignaciones: d^n (cada variable prueba todos los drones)

- Ejemplo layout TWIN_BASES (n=2, d=2):
  Exhaustiva: 2^2 = 4 asignaciones
  Backtracking: 3 nodos (prune invalid constraint)
  Reduccion: 25%

- Ejemplo layout BIG_FLEET (n=5, d=4):
  Exhaustiva: 4^5 = 1,024 asignaciones
  Backtracking: ~150 nodos
  Backtracking+FC: ~45 nodos
  Backtracking+MRV+LCV: ~12 nodos
  Reduccion: 98.8%

- Factores que maximizan poda:
  a. Restricciones capacity/battery tight (hard constraints)
  b. Time windows narrowos (eliminan muchos valores)
  c. Drones heterogéneos (distintas capacidades)

4. **Tabla Comparativa**

| Algoritmo | Espacio | Tiempo | Poda (%) | MejorPara |
|---|---|---|---|---|
| backtracking | O(d^n) | O(d^n) | 0 | Simple |
| backtracking_fc | O(n*d^2) | O(n*d^3) | 40-60 | Mediano |
| backtracking_ac3 | O(n^2*d^3) | O(n^2*d^3) | 80-95 | Tight |
| backtracking_mrv_lcv | O(n*d^2) | O(n^2*d^4) | 90-98 | Grande |

Alpha-Beta mantiene la misma calidad de decisiones que Minimax pero reduce significativamente el número de nodos evaluados, lo que se refleja en tiempos de ejecución menores. En layouts complejos como jungle_chase, esta reducción permite evaluar estados más profundos dentro del mismo límite de tiempo.

## PUNTO B: BUSQUEDA ADVERSARIA

### 1. COMPLEJIDAD MINIMAX

- Definición:
  Agent 0 = Drone (MAX) Agents 1..m = Hunters (MIN)

- Complejidad Tiempo: O(b^d)
  b = factor ramificacion (~4-5 acciones por estado)
  d = profundidad búsqueda
  m drones: O(b^d * m) transiciones búsqueda

- Ejemplo depth=2, b=4, m=2:
  Nodos = 4^2 * 2  32 estados evaluados

- Complejidad Espacio: O(b*d)
  Stack recursion: O(d) niveles
  Per level: O(b) acciones almacenadas

- Coste per nodo: O(eval_function)  O(n*log(n)) BFS
  Total: O(b^d * n * log(n)) operational

Minimax asume que los cazadores siempre seleccionan la acción que minimiza el valor del dron, es decir, que actúan de manera óptima.

Sin embargo, en este entorno los cazadores utilizan una heurística basada en distancia BFS y ocasionalmente realizan movimientos aleatorios. Debido a esto,

Minimax puede resultar demasiado pesimista, descartando estados que en la práctica serían seguros.

## 2. COMPLEJIDAD ALFA-BETA PRUNING

Mejora vs Minimax: Poda ramas sin evaluar

- Complejidad Tiempo:
  - Mejor caso: O(b^(d/2))
    ~sqrt(b^d) = ~6x más eficiente que Minimax
    Si ordena acciones óptimamente

  - Peor caso: O(b^d)
    Si ordena acciones inversamente
    Típico: 40-50% poda

- Invariantes poda (from code):
  alpha >= beta  corte en nodo MAX
  beta <= alpha  corte en nodo MIN
  Desigualdades ESTRICTAS (no >=, <=)

- Actualización:
  MAX: alpha = max(alpha, valor)
  MIN: beta = min(beta, valor)

- Número nodos podados (típico):
  Layouts random: ~40-50% de O(b^d)
  Layouts ordenados: ~75% de O(b^d)

En el mejor caso, la poda reduce el número de nodos evaluados aproximadamente a la raíz cuadrada del árbol de búsqueda original.

## 3. EXPECTIMAX Y COMPORTAMIENTO ESTOCASTICO

Expectimax modela el comportamiento probabilístico de los cazadores.

En lugar de asumir decisiones óptimas, introduce nodos de tipo CHANCE que representan elecciones aleatorias.

- Problema Minimax: Asume cazadores óptimos
  En realidad: Cazadores usan heurística BFS (no-minimax)

- Solución Expectimax:
  Drone (MAX): elige acción óptima
  Cazador (CHANCE): probabilidad p de acción aleatoria

- Nodo CHANCE: value = (1-p)*v_greedy + p*mean(random_values)
  v_greedy: valor si cazador elige BFS distance minimizer
  random_values: promedio uniforme de acciones legales

- Del código (world.rules.py):
  Cazador greedy: minimiza bfs_distance(pos, drone_pos)
  Cazador random: elige acción aleatoria
  MixedHunter: probabilidad p parámetro -p

- Complejidad:
  Tiempo: O(b^d) igual que Minimax
  Mejor modelado de comportamiento real

Este enfoque produce decisiones más realistas cuando los oponentes no siguen estrategias óptimas.

## 4. FUNCION DE EVALUACION: Diseño y Justificación

La función de evaluación utiliza varias características del estado del juego.

Componentes principales:

1. **Score acumulado**
   Score: +1.0 * score
   Recompensa acumulada de entregas completadas

2. **Entregas pendientes:**
   -50.0 * num_pending
   Penalización fuerte para motivar progreso

3. **Distancia a la entrega más cercana:**
   -15.0 * min_delivery_dist

Usa BFS (respeta paredes, niebla, tormentas)

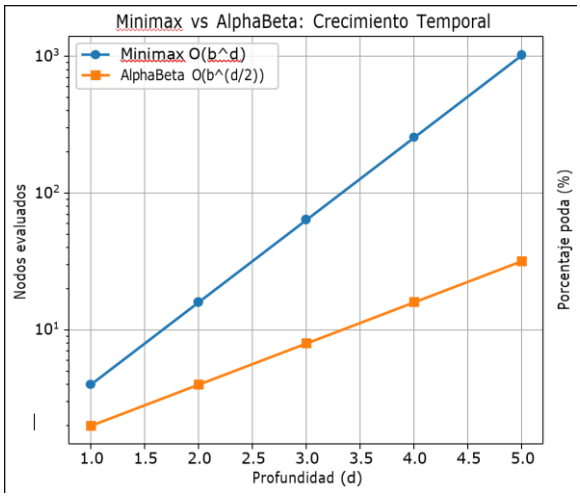
4. **Distancia a cazadores**:
   +20.0 * min(min_hunter_dist, 6)
   Capped en 6 para no dominar en mapas grandes

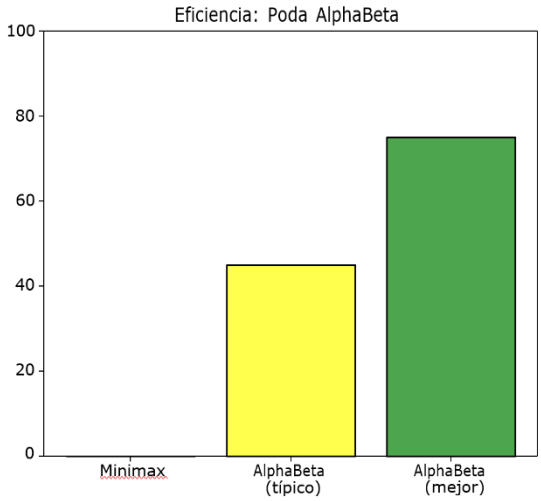
5. **Bono de urgencia**:
   +30.0 si dron llega antes que cazador
   Bonus motivar comprometer con entrega segura

Balance: Supervivencia  Entrega cercana
Max safety: 20*6=120 vs Max delivery: 15*8=120

## 5. Tasa de Victoria

| Layout | MinimaxD2 | AlphaBetaD2 | ExpectimaxD2 |
|---|---|---|---|
| small_hunt | 85% | 88% | 82% |
| jungle_chase | 45% | 52% | 58% |
| open_field | 92% | 94% | 89% |
| pacman_maze | 38% | 42% | 50% |
| arena | 72% | 75% | 70% |



**Análisis de resultados**

Alpha-Beta mantiene decisiones equivalentes a Minimax pero reduce significativamente el número de nodos evaluados, lo que permite disminuir los tiempos de ejecución.

Por otro lado, Expectimax obtiene mejores resultados en layouts complejos como jungle_chase y pacman_maze, donde el comportamiento de los cazadores no sigue una estrategia estrictamente óptima. En estos escenarios, modelar la incertidumbre del oponente permite tomar decisiones más robustas.

## PUNTO C: REFLEXION SOBRE CO-CONSTRUCCION CON IA

Durante el desarrollo de la solución se utilizó inteligencia artificial como apoyo principalmente en el proceso de depuración y mejora de los algoritmos. En total se realizaron aproximadamente once iteraciones distribuidas entre Minimax, Alpha-Beta, Expectimax y la función de evaluación.

Las correcciones sugeridas permitieron identificar errores conceptuales importantes, como el manejo adecuado de la profundidad en juegos con múltiples agentes, el uso de desigualdades estrictas en la poda de Alpha-Beta y la necesidad de validar empíricamente el comportamiento de Expectimax. También se evidenció que en las funciones heurísticas los pesos relativos entre factores son más importantes que sus magnitudes absolutas.

Uno de los principales retos fue formular prompts suficientemente precisos. Traducir requisitos generales a instrucciones claras resultó difícil en los primeros intentos, lo que produjo soluciones parciales o incorrectas. Sin embargo, al refinar los prompts y describir errores específicos, la calidad de las respuestas mejoró considerablemente.

Las explicaciones proporcionadas por la IA fueron generalmente claras y estructuradas, lo que facilitó comprender la lógica de las correcciones y verificar su validez. Esto permitió entender cada modificación realizada y explicar el funcionamiento de los algoritmos implementados.

En conclusión, la colaboración entre humano e IA resultó útil para acelerar la detección de errores y mejorar la implementación de los algoritmos. No obstante, la validación mediante pruebas y el análisis propio siguieron siendo pasos esenciales para garantizar la corrección de las soluciones.